KING SAUD UNIVERSITY

College of Computer and Information Sciences

CSC 212 | DATA STRUCTURES

Ride-Sharing System

Phase 1 Report: Worst-Case Time Complexity Analysis

GitHub: Alfarouq-Alshukri/CSC212-RideSharing-System

PROJECT TEAM AND DIVISION OF WORK

Team member	Student ID	Contribution
Khalid Alhazlool	446102664	Completed ride-related code
Alfarouq Alshukri	446109472	Completed driver-related code
Khalifa Alhashil	446104726	Completed rider-related code

1. Project overview

The system manages riders, drivers, private rides, and shared rides. It loads CSV files, supports searches and alphabetical listings, checks scheduling conflicts, and updates ride histories when rides are added or removed.

`Main` handles input and output. `RideSharingSystem` coordinates the operations. `RiderList`, `DriverList`, and `RideList` store records using the course's custom singly linked list. Riders and drivers have an additional name index, while rides stay ordered by pickup location. These maintained orders allow alphabetical retrieval in linear time.

2. Analysis assumptions and symbols

All results describe the worst case. A search may reach the last node or find no match. Methods with the same algorithm are grouped, and inherited methods are analyzed under their base class.

Symbol	Meaning
R, D, Q	Number of stored riders, drivers, and rides
P	Number of participants in one shared ride or scheduling request
S	Total participant entries across all stored shared rides
n	Number of nodes in the particular list being analyzed
H	Number of entries in one person's ride history; $H \le Q$

Main counting model: IDs, dates, references, and bounded-length text fields take constant work to read or compare. The number of records and participants can grow. Input parsing is counted separately by the number of characters read. Section 10 explains the effect of unbounded text lengths.

3. Calculation method used

For each algorithm, count the work, form the growth function, and simplify:

1. Assign S/E = 1 to each constant-time statement; declarations and braces have S/E = 0.
2. Count the execution frequency. A loop with n iterations normally checks its condition n + 1 times, including the final failed check.
3. Calculate Total = S/E × Frequency, then add the totals to obtain f(n).
4. Drop constant factors and lower-order terms to obtain Big O.

The following tables count the simplified pseudocode using one step per constant-time statement. Later tables use growth expressions with constant factors omitted. A helper that scans a list must contribute its full traversal cost.

3.1 One traversal: linear growth

This is the counting pattern for `DriverList.size()`, searches, and copying an already ordered list.

Statement	S/E	Frequency	Total
count = 0	1	1	1
for each of n nodes	1	n + 1	n + 1
count = count + 1	1	n	n
return count	1	1	1

$$f(n) = 1 + (n + 1) + n + 1 = 2n + 3$$

The highest-order term is n, so O(n). For $n \geq 1$, $2n + 3 \leq 5n$; choose c = 5, n0 = 1.

3.2 Dependent nested loops: quadratic growth

Shared scheduling compares each candidate rider ID with the earlier IDs. With P distinct IDs, no duplicate causes an early return.

Statement	S/E	Frequency	Total
for i = 0 to P - 1	1	P + 1	P + 1
for j = 0 to i - 1	1	1 + 2 + ... + P	P(P + 1)/2
compare IDs i and j	1	0 + 1 + ... + (P - 1)	P(P - 1)/2
return no duplicates	1	1	1

$$f(P) = (P + 1) + P(P + 1)/2 + P(P - 1)/2 + 1$$

$f(P) = P^2 + P + 2$, so $O(P^2)$. For $P \geq 1$, $P^2 + P + 2 \leq 4P^2$; choose c = 4, P0 = 1.

Sequential work adds: two scans give $n + n = 2n$, hence O(n). Repeated work multiplies: n scans of up to n nodes give $n \times n = n^2$, hence $O(n^2)$. For nested loops whose lengths change, use a sum as above.

4. Basic list and entity methods

4.1 Constant-time methods

These methods do a fixed amount of work, so `f(n) = k` for a constant k, giving O(1).

Class	Methods covered
LinkedList	Constructor, empty, last, full, findFirst, findNext, retrieve, update, insert
Node	Both constructors
RiderList, RideList	Constructors and size
DriverList, RideSharingSystem	Constructors
DateTime	Numeric and array constructors, daysInMonth, all getters, compareTo, format, toString
Person	Constructor, getId, getName, getPhoneNumber, setPhoneNumber, getRideHistory
Rider	Constructor, getEmail, setEmail, getHomeCity, setHomeCity, compareTo
Driver	Constructor, getVehiclePlate, setVehiclePlate, getVehicleType, setVehicleType, compareTo
Ride	Constructor, all getters, setDropoffLocation, setDriver, compareTo
PrivateRide	Constructor, getRider, setRider, hasRider
SharedRide	Constructor and isEmpty

Constructors initialize fields and empty lists. `getRideHistory()` returns a reference. Phone and plate validation checks at most 10 and 7 characters.

4.2 LinkedList.remove() and DriverList.size()

`LinkedList.remove()` searches for the current node's predecessor, even if the caller already found that node. `DriverList.size()` traverses its entire chain.

Method	Growth calculation	Big O
LinkedList.remove	Fixed work + at most n - 1 predecessor advances	O(n)
DriverList.size	One traversal: f(D) = 2D + 3 in the simplified model	O(D)

`RiderList.size()` and `RideList.size()` return maintained counters, so they remain O(1).

4.3 Changes that refresh an index

Under the normal system structure, each person has one name index and each ride belongs to one main ride list.

Method	Work added together	Big O
Person.setName	Validate + unlink from name index + ordered reinsertion	O(R) for a rider; O(D) for a driver
Ride.setPickupLocation	Validate + find/remove ride + ordered reinsertion	O(Q)
Person.attachNameIndex; Ride.attachList	Prepend one registration node	O(1)
Person.detachNameIndex; Ride.detachList	Scan owner registrations	O(1) with one owner

For a rider rename, `R + R + 1 = 2R + 1`, giving O(R). Before registration, these setters take O(1).

5. Rider and driver lists

Use n = R for `RiderList` and n = D for `DriverList`. The following growth expressions count the main traversals; fixed factors do not affect Big O.

Methods	Growth calculation	Big O
add	Uniqueness search + ID-order scan + name-index insertion: n + n + n	O(n)
findById, findByEmail, findByName, findByHomeCity, findByVehiclePlate, findByVehicleType, where defined	Scan n records; collecting k matches adds k ≤ n: n + k	O(n)
removeById, removeByEmail, removeByVehiclePlate, where defined	Search + name-index removal + primary-list removal: n + n + n	O(n)
getAll, getAllAlphabetically	Copy n references using constant-time result insertions: n	O(n)
removeByName, removeByHomeCity, removeByVehicleType, where defined	Outer scan + at most n removals, each costing up to n: n + n^2	$O(n^2)$

The unique-key removals stop after one match. The name, city, and vehicle-type removals must remove every match. A long retained prefix followed by many matching records causes repeated scans: about n/2 removals each revisiting about n/2 nodes gives $n^2/4$, so the worst case is quadratic.

Alphabetical retrieval does not sort records again. It copies the maintained name index, so its growth is linear.

5.1 PersonNameIndex helpers

Methods	Growth calculation	Big O
compare	Compare bounded name fields, then IDs if needed: 1	O(1)
add, insert	Find ordered position: n	O(n)
unlink, remove	Find one identity and unlink it: n	O(n)
nameChanged	Unlink, then reinsert: n + n	O(n)
getAll	Copy the maintained chain: n	O(n)

5.2 System methods that delegate to these lists

A wrapper adds a fixed call and return. Its growth is `1 + f_list`, so it keeps the list method's Big O.

RideSharingSystem methods	Big O
addRider, searchRiderById, searchRiderByEmail, searchRidersByName, searchRidersByHomeCity, getAllRiders	O(R)
addDriver, searchDriverById, searchDriverByVehiclePlate, searchDriversByVehicleType, getAllDrivers	O(D)

These listing bounds cover building the result list. Formatting and printing its objects are analyzed in Section 10.

6. Ride lists and shared participants

6.1 RideList operations

Methods	Growth calculation	Big O
containsId, insertRide	One ID or ordered-position scan: Q	O(Q)
addRide	Duplicate-ID scan + ordered insertion: Q + Q	O(Q)
removeRideById	ID search + predecessor search: Q + Q	O(Q)
pickupLocationChanged	Find + remove + reinsert: Q + Q + Q	O(Q)
findByPickupLocation	Compare each pickup and collect matches: at most Q + Q	O(Q)
getAllAlphabetically	Copy the already ordered chain: Q	O(Q)
findByRiderName	Visit Q rides and examine shared participants: Q + S	O(Q + S)
rideHasRiderNamed	Private: one name; shared: copy and scan P participants	O(1) private; O(P) shared

`RideSharingSystem.searchRidesByPickupLocation`, `searchRidesByRiderName`, and `getAllRidesAlphabetically` have the same bounds as their corresponding list operations.

6.2 SharedRide participant operations

Method	Growth calculation	Big O
getParticipants	Copy P references: P	O(P)
hasRider	Search up to P participant IDs: P	O(P)
addParticipant	Duplicate search + walk to tail: P + P	O(P)
removeParticipantById	Search + predecessor search: P + P	O(P)

`Ride.hasRider()` is abstract. Its implementation is O(1) in `PrivateRide` and O(P) in `SharedRide`.

6.3 Unique shared participants at a pickup location

`getSharedRideParticipants()` first finds matching rides, then processes their shared participants. Let K be the total participant entries at that pickup and U the number of distinct riders returned, with U ≤ min(R, K).

Each `addParticipantOnce()` call searches the result built so far, costing at most O(U). Therefore the growth bound is:

```
f(Q, K, U) = Q + K + KU
```

Big O: O(Q + K(U + 1)). If all K entries are different, the duplicate-check lengths grow as `0 + 1 + ... + (K - 1) = K(K - 1)/2`, giving O(Q + K^2). The helper itself is O(U + 1).

7. Scheduling rides

7.1 Conflict detection

`validRideDetails()` checks endpoints and time order in O(1) under the stated field model. `conflicts()` copies the Q stored rides and checks overlapping intervals. For each overlap, it may check all P proposed riders against the stored ride's participants.

Work	Frequency or size	Growth contribution
Copy rides and compare time intervals	Q rides	Q
Candidate-rider checks for private rides	Up to P per ride	At most PQ
Candidate-rider checks for shared rides	P scans of each shared participant list	PS

`f_conflict = Q + PQ + PS`, giving O(Q + P(Q + S)).

For a private request, P = 1, giving O(Q + S). Back-to-back rides do not conflict: overlap requires `newStart < oldEnd` and `oldStart < newEnd`.

7.2 schedulePrivateRide()

Stage	Growth contribution
Find rider and driver	R + D
Validate and construct	1
Check conflicts	Q + S
Insert into the ride list	Q
Append to rider and driver histories	H_r + H_d + 1

Add the stages. Since each history has at most Q entries:

```
f_private = R + D + 4Q + S + 2
```

Dropping fixed factors and constants gives O(R + D + Q + S).

7.3 scheduleSharedRide()

Stage	Growth contribution
Find driver and P riders	D + PR
Check duplicate candidate IDs	P(P - 1)/2
Build participants through repeated addParticipant calls	P^2
Check conflicts	Q + P(Q + S)
Insert ride and append all histories	Q + (P + 1)(Q + 1)

The two participant-building stages follow the triangular-loop calculation in Section 3. Adding these contributions gives the upper-bound growth expression:

```
f_shared = D + PR + P² + P(Q + S) + Q + P + 1
```

For P ≥ 1, Q is covered by PQ and P by P^2. Thus O(D + PR + P^2 + P(Q + S)). For a valid set of distinct registered riders, P ≤ R, so P^2 is also covered by PR, yielding O(D + PR + P(Q + S)) on that domain.

8. Histories and cascade removal

8.1 History helpers and deleting one ride

History lists have no tail pointer. Appending requires walking to the end; removing an entry requires a search and possibly another predecessor scan.

Method	Growth calculation	Big O
addHistory	Walk H entries, then insert: H + 1	O(H + 1)
removeHistory	Find entry + remove current: H + H + 1	O(H + 1)
deleteRide	Remove global ride + driver history + participant copy and rider histories	O(Q + P + H_d + sum(H_r))

For `deleteRide()`, use P = 1 for a private ride and sum over its one rider. For a shared ride, sum over all its participants. Since H ≤ Q, a convenient general bound is O((P + 1)(Q + 1)).

8.2 removeRider()

The system finds the rider, checks stored rides, deletes their private rides, and removes their shared memberships. Shared rides are deleted only if they become empty.

Stage	Growth contribution
Find and finally remove rider	R
Copy rides and check memberships	Q + S
Remove shared memberships	At most S
Update histories and delete affected rides	At most Q operations, each up to Q: Q^2

```
f_removeRider = R + Q + 2S + Q²
```

Since Q ≤ Q^2 for Q ≥ 1, the result is O(R + S + Q^2). Other riders' histories remain valid when the shared ride survives, because they still refer to that ride.

8.3 removeDriver()

Removing a driver deletes all their rides. Every deleted shared ride also requires removing one history entry for each participant.

Stage	Growth contribution
Find and finally remove driver	D
Copy and inspect stored rides	Q
Global ride and driver-history removals	At most Q × Q = Q^2
Private-rider history removals	At most Q × Q = Q^2
Shared-rider history removals	At most S × Q = QS

```
f_removeDriver = D + Q + 2Q² + QS
```

The result is O(D + Q^2 + QS). The extra QS term comes from updating participant histories. These system removals include more work than removing one record directly from `RiderList` or `DriverList`.

9. Loading CSV files

Let C be the number of processed rows, B the number of processed input characters, and N0 the number of people already stored before an import.

9.1 loadRidersFromCSV() and loadDriversFromCSV()

Reading the file is not the only work. Each successful row calls an insertion method on a list that is growing.

Work	Frequency	Growth contribution
Read, split, trim, and parse	B characters	B
Insert row 1	Once	N0 + 1
Insert row 2	Once	N0 + 2
...	...	...
Insert row C	Once	N0 + C

```
f_load = B + (N0 + 1) + (N0 + 2) + ... + (N0 + C)

f_load = B + CN0 + C(C + 1)/2
```

Therefore $O(B + CN0 + C^2)$. From an empty list, with bounded row length so $B = O(C)$, this becomes $O(C^2)$. Use $C = C_R$ for riders and $C = C_D$ for drivers. The `columns()` helper takes $O(b)$ for a row containing b characters.

9.2 loadRidesFromCSV()

Each row invokes private or shared scheduling. For row i, let Q_i and S_i be the stored ride and shared-participant counts immediately before scheduling, and let P_i be its rider count. Use $P_i = 1$ for private rows.

Define the common scheduling growth bound:

```
G_i = D + P_i R + P_i² + P_i(Q_i + S_i) + Q_i + 1
```

Parsing all rows adds B. Thus:

```
f_loadRides = B + sum(G_i), for i = 1,...,C
```

Big O: $O(B + \text{sum}(G_i))$. This keeps the cost of participant counts that may vary between rows.

For a simpler special case, suppose each row has at most a fixed number of participants. Then $Q_i = Q0 + i - 1$ and $S_i \le S0 + (i - 1)Pmax$. Summing the growing i terms gives:

$O(B + C(R + D + Q0 + S0 + 1) + C^2)$. With fixed starting populations and bounded row lengths, this is $O(C^2)$.

CSV loading stops at the first invalid row and retains earlier valid rows. A fully valid file allows the maximum number of insertions. The time complexity is determined by how the algorithms scale with input size, rather than by the size of the current dataset.

10. Formatting, input, and remaining costs

10.1 String conversion and printing

Printing a record can include its history and shared participants. Repeated concatenation of immutable strings copies the growing prefix.

Methods	Growth calculation	Big O
Ride.toString, PrivateRide.toString	Fixed number of bounded fields	O(1)
SharedRide.toString	Copy growing participant text: 1 + 2 + ... + P	$O((P + 1)^2)$
Person.toString, Rider.toString, Driver.toString	Format each history ride and repeatedly copy the accumulated description	See below

For H history entries, let p_i be the participant count of entry i; use p_i = 1 for private rides. A growth bound for person formatting is:

```
f_person = 1 + sum((p_i + 1)²)

+ sum((H - i + 1)(p_i + 1)), for i = 1,...,H
```

The first sum covers ride formatting. The second covers repeated copying of the history text. The result is O(f_person); with only private rides, it simplifies to $O((H + 1)^2)$.

`Main.printList()` costs O(n + sum(F_i + output_i)), where F_i is each object's formatting cost and output_i is its character count. Linear result retrieval therefore does not guarantee linear full display time.

10.2 Parsing and Main

Methods	Growth calculation and Big O
DateTime(String), DateTime.parse	Scan b input characters: O(b)
Main.read, required, number, vehicleType	Read/validate b input characters: O(b)
Main.printMenu, printResult	Fixed messages: O(1)
Main.addRider, addDriver	Input + insertion: O(b + R), O(b + D)
Main.addRide	Input + selected scheduling cost
Main.main	Three startup loads + the sum of input, operation, and output costs for completed commands

An interactive session has no fixed command count. Its total time must include the number of commands; human waiting time is excluded.

10.3 If text length or ownership can grow

The main tables use bounded fields and one normal owner. With maximum field length L, text comparisons and validation can cost O(L): person insertion becomes O((n + 1)L), pickup search O(QL), and rider-name ride search O((Q + S)L). String-formatting bounds gain a factor L. Parsing already counts input characters.

For multiple owners, a rename or pickup change refreshes every attached index/list. Sum the corresponding per-owner traversal costs. Detaching an owner takes O(number of owner registrations).

10.4 Conclusion

Most searches, insertions, and ordered retrievals are linear. Repeated deletions, growing CSV imports, and participant-building loops can be quadratic. Shared scheduling and driver removal also depend on the total shared-participant count S.